

CHAPTER: 1

★ "INTRODUCTION TO DSA" ★

DATA STRUCTURES:-

Definition:-

"The shape in which data is arranged in the memory and its access is called DATA STRUCTURES."

Explanation:-

By now you have enough knowledge of programming fundamental and object oriented programming to get into data structures.

Up until now, you have not really bothered with how data is arranged in the memory, but in data structures, this is all we are gonna do.

EXAMPLES:-

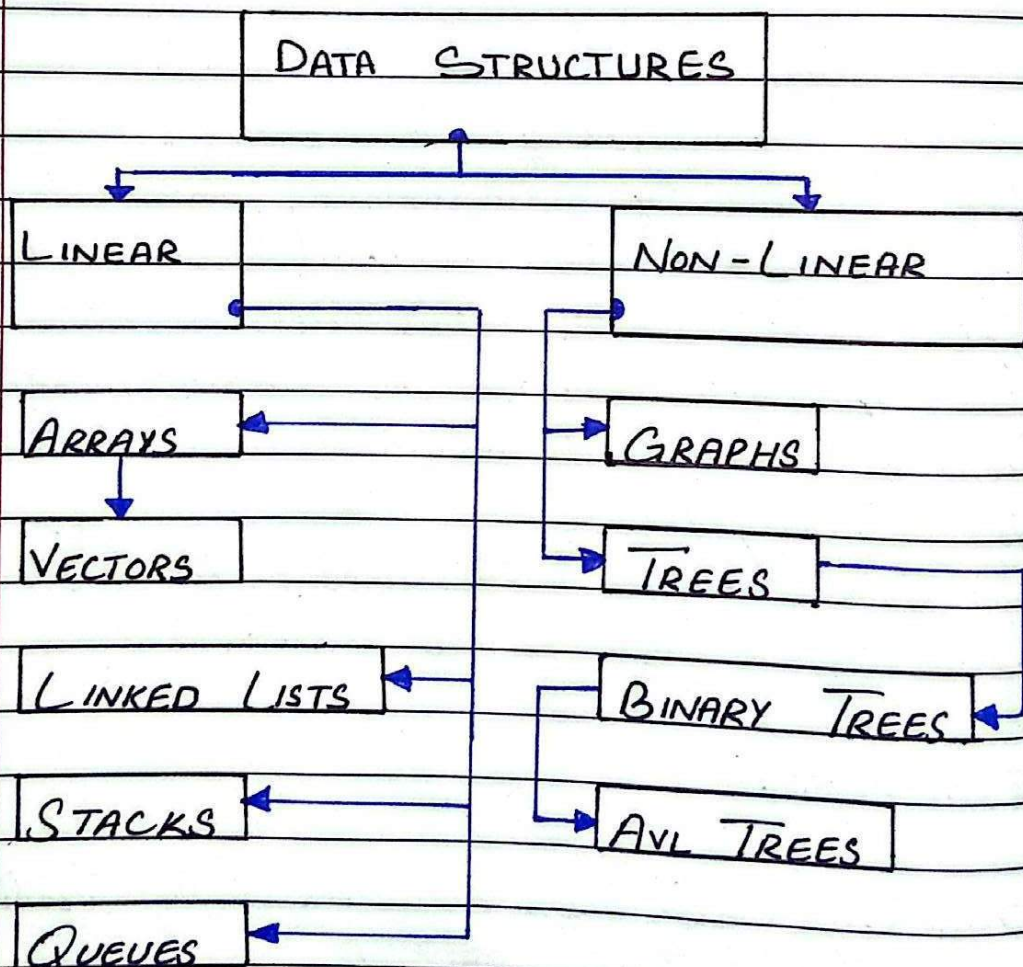
Following are some commonly used data structures, which we will be studying in the coming chapters:-

Date: / / Day

- | | |
|------------|-----------------|
| 1) Arrays | 5) Linked Lists |
| 2) Vectors | 6) Trees |
| 3) Stack | 7) Graphs |
| 4) Queues | etc ... |

Out of these, some are called linear and others are non linear. Each data structure has its own use and importance.

CATEGORIZATION DIAGRAM:-



ALGORITHMS :-

Definition :-

"The specific sequence of steps performed to achieve a result is called an ALGORITHM."

Explanation :-

The proper procedure that us from the beginning of a task to its end is basically an algorithm, consider comparing two numbers to find the bigger one, its algorithm would be like:

Step 1: Read two numbers from input.

Step 2: If $n1 < n2$, set $large = n2$,
otherwise, set $large = n1$.

Step 3: Write $large$ to output.

EXAMPLES :-

There are various examples of algorithms but for now, we will look at some come as beginner level algorithms:-

Date: ___/___/___

- 1) Linear Search
- 2) Binary Search
- 3) Euclidean Algorithm.
- etc...

LINEAR SEARCH:-

It is basically an algorithm which is used to search for an element in an array. It compares the target element with the value at each index, and upon finding a match, it returns the index. If the element is not found till the end of array, it returns -1 which can never be an index. -1 shows that element is not present in the array, let's look at the code now:-

Code:-

```
int linearSearch(const int arr[], int, int t)
{
    for(int i=0; i<S; i++)
        if (arr[i]==t) return i;
    return -1;
}
```


BINARY SEARCH:-

There is a faster searching algorithm that can only be used if the given array is sorted. This algorithm is called "Binary Search". Some people call it "Logarithmic Search". We will find the reason for this name in the chapter of "Time & Space Complexity". Let's look at binary search for now.

It looks for the target value by comparing the mid value of array with target. If mid is less than target, left half of array is discarded and right half is checked. But if mid is greater than target, right half of array is discarded and left half is checked. This loops until mid is equal to target or the array finishes and value isn't found. If value of target appears at mid, its index is returned.

But if the array finishes and element is not found, then just like linear search -1 is returned as an indication that target is not present in the array.

Date: / / Day

Now let's look at the code.

Code :-

```
int binarySearch(const int arr[], int s, int t)
{
    int start = 0, end = s-1, mid;
    while (start <= end)
    {
        mid = start + (end - start) / 2;
        if (arr[mid] == t) return mid;
        if (arr[mid] < t) start = mid + 1;
        if (arr[mid] > t) end = mid - 1;
    }
    return -1;
}
```

EUCLIDEAN ALGORITHM:-

Euclidean algorithm is a calculative algorithm used for finding the greatest common divisor or "GCD" of two numbers. It works by subtracting the smaller number from the larger number until both get equal. This equal number is basically the gcd. For now, let's look at the code :-

Date: 1/1

Day

Code:-

```
int gcd(int a, int b)
{
    while (a != b)
    {
        if (a < b) b -= a;
        else      a -= b;
    }
    return a;
}
```

This is gonna be all for the introduction. All the codes given in this chapter can be found in my github repository at:

"github.com/coder-Rebro/Fast-University"
Go into DSA > Algorithms.

Next up, we are gonna take some dive into pointers and arrays. Till Then Good Luck!